PROJET PYTHON – MODULE TURTLE

Exercice 2 : La spirale colorée

Analyse du code et réponses aux questions de réflexion

Nom	Muteba Wa Muteba
Prénom	Israël
Classe	L2 (CS)
Fichier analysé	`projet(exo2).py`
Date	12 novembre 2026

1. Présentation de l’exercice

Ce programme dessine, sur fond noir, une **spirale carrée** de 600 côtés. À chaque étape, la tortue change de couleur, avance d’une distance un peu plus grande que la précédente, puis tourne de 90° vers la gauche. C’est la combinaison d’une **longueur qui augmente** et d’un **angle qui reste fixe** qui crée la spirale.

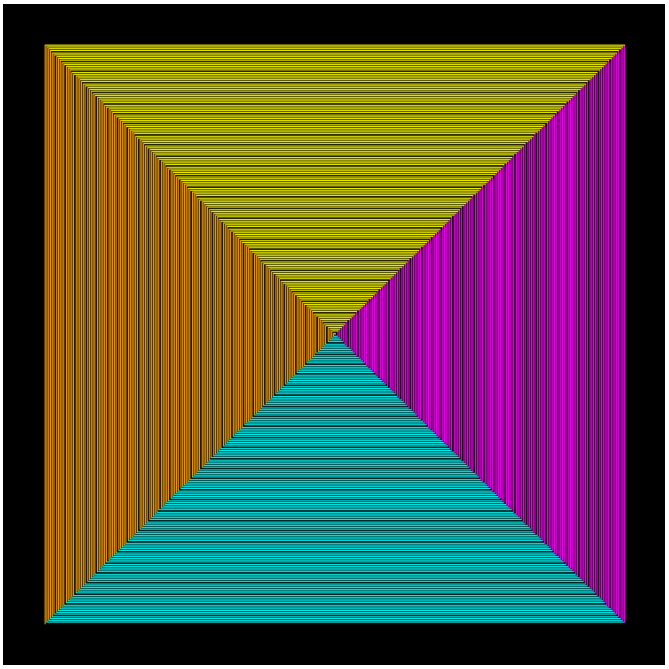


Figure 1 – *Résultat du programme : 600 côtés, angle de 90°, coefficient de 1,2 (illustration)*

Observation : avec 4 couleurs et un angle de 90° (donc 4 côtés par tour), chaque direction garde toujours la même couleur : cyan vers la droite, magenta vers le haut, yellow vers la gauche et orange vers le bas. C’est pourquoi la spirale apparaît divisée en quatre zones colorées. De plus, les lignes sont très serrées (2,4 pixels d’écart entre deux tours), ce qui donne l’impression d’un carré rempli.

2. Code source analysé

Fichier : projet(exo2).py

```
import turtle

t = turtle.Turtle()
t.speed(0)
turtle.bgcolor("black")
couleurs = ["cyan", "magenta", "yellow", "orange"]

for i in range(600):
    t.color(couleurs[i % len(couleurs)])
    t.forward(i * 1.2)
    t.left(90)

turtle.done()
```

Fonctionnement pas à pas

1. **Préparation** : création de la tortue `t`, vitesse maximale avec `t.speed(0)` et fond noir avec `turtle.bgcolor("black")`.
2. **Liste de couleurs** : les quatre noms de couleurs sont rangés dans la liste `couleurs`.
3. **Boucle** : `for i in range(600)` répète le bloc 600 fois, avec `i` qui prend les valeurs 0, 1, 2, ... 599.
4. **Couleur** : `couleurs[i % len(couleurs)]` choisit une couleur qui revient au début après la dernière.
5. **Avance** : `t.forward(i * 1.2)` trace un côté de plus en plus long (le dernier mesure environ 719 pixels).
6. **Rotation** : `t.left(90)` tourne la tortue d'un quart de tour vers la gauche.
7. **Fin** : `turtle.done()` garde la fenêtre ouverte.

Les six premières étapes de la boucle donnent :

`i`	Couleur	Longueur du côté (`i * 1.2`)	Direction
0	cyan	0 (rien n'est tracé)	vers la droite
1	magenta	1,2	vers le haut
2	yellow	2,4	vers la gauche
3	orange	3,6	vers le bas
4	cyan	4,8	vers la droite
5	magenta	6,0	vers le haut

3. Réponses aux questions d'analyse

Question 1 – Quelles instructions sont répétées ? Pourrait-on utiliser une boucle ?

Les trois instructions `t.color(...)`, `t.forward(...)` et `t.left(90)` sont répétées **600 fois**. Seule la valeur de `i` change d'une répétition à l'autre. Sans boucle, il faudrait écrire 1 800 instructions :

Sans boucle (impraticable)

```
t.color("cyan");    t.forward(0 * 1.2);   t.left(90)
t.color("magenta"); t.forward(1 * 1.2);   t.left(90)
t.color("yellow");  t.forward(2 * 1.2);   t.left(90)
t.color("orange");  t.forward(3 * 1.2);   t.left(90)
t.color("cyan");    t.forward(4 * 1.2);   t.left(90)
# ... et ainsi de suite jusqu'à i = 599
```

La boucle `for` du programme est donc la solution adaptée : elle réduit ces 1 800 instructions à 4 lignes.

Il existe aussi une autre façon de répéter, avec **deux boucles imbriquées**. Chaque tour complet de la spirale comporte 4 côtés (un par couleur), donc 150 tours suffisent pour 600 côtés :

Variante avec deux boucles

```
longueur = 0
for tour in range(150):
    for couleur in couleurs:
        t.color(couleur)
        t.forward(longueur)
        t.left(90)
        longueur += 1.2
```

Cette variante dessine exactement la même spirale. Elle évite le modulo, mais le nombre total de côtés dépend alors du nombre de couleurs (150 × 4) : si l'on ajoute une couleur, la spirale compte plus de côtés.

Question 2 – Une partie du dessin pourrait-elle être isolée dans une fonction ?

Dans le programme actuel, aucune fonction n'est utilisée : tout le dessin est dans la boucle. On peut pourtant isoler **la spirale entière** dans une fonction qui reçoit ses réglages en paramètres.

Proposition : la fonction spirale()

```
def spirale(nombre_cotes, coefficient, angle, couleurs):
    for i in range(nombre_cotes):
        t.color(couleurs[i % len(couleurs)])
        t.forward(i * coefficient)
        t.left(angle)

spirale(600, 1.2, 90, ["cyan", "magenta", "yellow", "orange"])
```

Les avantages sont les suivants :

- **Réutilisation** : on peut dessiner plusieurs spirales différentes en appelant la fonction avec d'autres valeurs (après avoir déplacé la tortue avec `t.penup()`, `t.goto(x, y)` et `t.pendown()`).
- **Lisibilité** : la dernière ligne résume tout le dessin en un seul appel, avec des paramètres nommés.

- **Expérimentation facile** : pour tester un autre angle ou une autre taille, on ne modifie que les valeurs de l'appel.

On pourrait aussi isoler un seul côté dans une fonction `cote(i)`, mais le gain serait faible car le corps de la boucle ne contient que trois instructions.

Question 3 – Quels paramètres pouvez-vous modifier pour changer le résultat ?

Paramètre	Rôle	Effet sur le dessin
`range(600)`	Nombre de côtés	Plus la valeur est grande, plus la spirale est grande et dense. Avec 600 côtés, le dernier côté mesure environ 719 pixels, soit souvent plus que la fenêtre par défaut.
`1.2`	Coefficient de croissance	Chaque côté est 1,2 pixel plus long que le précédent. L'écart entre deux tours vaut `2 × 1.2 = 2,4` pixels : plus le coefficient est grand, plus la spirale est aérée.
`90`	Angle de rotation	90° donne une spirale carrée, 120° un triangle. Un angle proche de 60° ou de 90° (59°, 91°) donne un tourbillon.
`couleurs`	Liste de couleurs	On peut ajouter, retirer ou changer des couleurs. Avec 4 couleurs et 90°, chaque direction garde sa couleur ; avec 5 couleurs, les couleurs changent d'un tour à l'autre dans chaque direction.
`t.speed()`, `bgcolor()`	Vitesse et fond	Vitesse du tracé et couleur de l'arrière-plan.

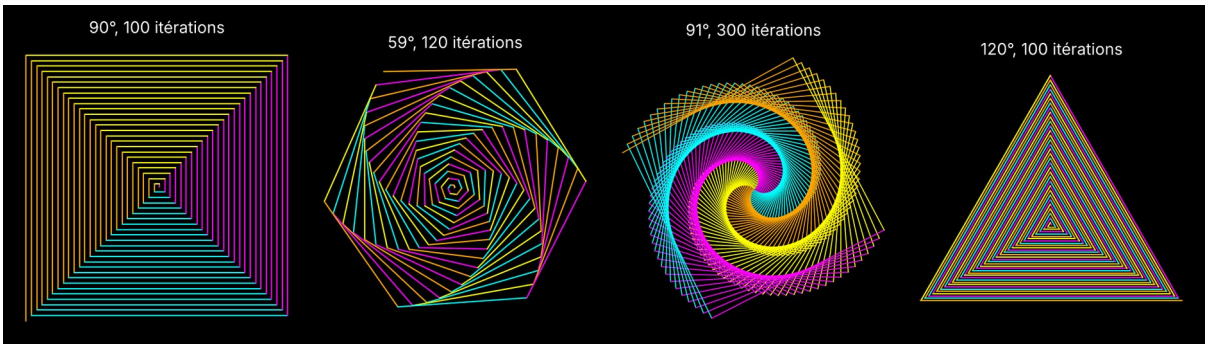


Figure 2 – *Effet de l'angle et du nombre de côtés : 90° (100 côtés), 59° (120 côtés), 91° (300 côtés), 120° (100 côtés)*

L'angle est le paramètre qui change le plus la forme. Un angle de **360 divisé par un entier** (90 = 360 / 4, 120 = 360 / 3) donne un polygone régulier qui se referme ; un angle légèrement différent (59°, 91°) fait « glisser » chaque côté par rapport au précédent et crée l'effet de tourbillon.

Question 4 – Quelles nouvelles instructions ou notions avez-vous dû découvrir ?

Notion	Explication	Exemple dans le code
Module `turtle`	Bibliothèque de dessin pilotée par une « tortue »	`t = turtle.Turtle()`

Notion	Explication	Exemple dans le code
Méthodes de la tortue	`forward()` avance, `left()` tourne, `color()` change la couleur, `speed()` règle la vitesse, `bgcolor()` colore le fond	`t.forward(i * 1.2)`
Liste Python	Plusieurs valeurs rangées sous un même nom et accessibles par un indice qui commence à 0	`couleurs[0]` donne `"cyan"`
`len(liste)`	Nombre d'éléments de la liste : le code reste correct si l'on ajoute des couleurs	`len(couleurs)`
Opérateur modulo `%`	Reste d'une division : fait revenir l'indice à 0 après le dernier élément	`i % 4`
Variable de boucle	`i` change à chaque répétition et sert dans les calculs	`i * 1.2`
Nombres décimaux	Les distances peuvent être des nombres à virgule (notés avec un point)	`1.2`

Le modulo est la notion clé qui permet de répéter les 4 couleurs indéfiniment :

`i`	0	1	2	3	4	5	6	7
`i % 4`	0	1	2	3	0	1	2	3
Couleur	cyan	magenta	yellow	orange	cyan	magenta	yellow	orange

Il faut aussi comprendre le **principe géométrique de la spirale** : en répétant « avancer d'une longueur croissante puis tourner d'un angle constant », la tortue s'éloigne peu à peu de son point de départ en décrivant des tours de plus en plus larges.

Question 5 – Qu'avez-vous essayé lorsque le résultat n'était pas celui attendu ?

Problème observé	Hypothèse ou essai	Solution retenue
`IndentationError: expected an indented block after 'for' statement`	Python ne reconnaissait pas les trois lignes suivantes comme faisant partie de la boucle	Ajout de 4 espaces devant chaque ligne du corps de la boucle
Première version avec `left(59)` et 120 côtés : spirale en étoile	Comparer plusieurs angles (59°, 90°, 91°, 120°) pour voir leur effet	Choix de `left(90)` et augmentation à 600 côtés pour une spirale carrée
Dessin très dense et très grand avec 600 côtés	Essayer d'abord avec 100 côtés pour observer l'effet de chaque paramètre	Régler `range()` et le coefficient selon la taille souhaitée
Fichier de travail nommé `is.py`	`is` est un mot réservé de Python : un tel nom de fichier pose problème dès qu'on veut l'importer	Choisir un nom sans mot réservé (par exemple `spirale.py`)

Voici l'erreur d'indentation et sa correction :

Message d’erreur affiché par Python

```
IndentationError: expected an indented block after 'for' statement on line 8
```

Avant (erreur)

```
for i in range(600):
t.color(couleurs[i % len(couleurs)])
t.forward(i * 1.2)
t.left(90)
```

Après (corrigé)

```
for i in range(600):
    t.color(couleurs[i % len(couleurs)])
    t.forward(i * 1.2)
    t.left(90)
```

Méthode suivie : lire le message d’erreur (il indique le type d’erreur et le numéro de ligne), changer **une seule valeur à la fois**, tester avec de petites valeurs avant de passer aux grandes, puis comparer les résultats.

> **Remarque pratique** : sous Kali Linux, si la fenêtre turtle ne s’ouvre pas, il faut vérifier que la bibliothèque graphique Tkinter est installée avec la commande `sudo apt install python3-tk`.

4. Conclusion

Cet exercice montre qu’un dessin spectaculaire peut tenir en quatre lignes grâce à une **boucle**, une **liste de couleurs** et l’opérateur **modulo**. J’ai compris que la forme finale dépend surtout de deux réglages : la façon dont la longueur augmente et l’angle de rotation. Isoler le dessin dans une fonction `spirale()` permettrait de réutiliser facilement le même code avec d’autres paramètres.

Document rédigé par **Muteba Wa Muteba Israël** – Classe L2 (CS).